PRACTICAL 1

MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options.

1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator.

2. Create a public class named MiniBank that contains the main method.

3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header.

4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer.

5. Display the numbered menu to the user and read the chosen number with a Scanner.

6. Use a switch expression on the choice to print a short placeholder for that option (for example, "Deposit — to be implemented in a later lab"). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message.

CODE:

```
import java.util.Scanner;

ublic class MiniBank {

  // Record for Bank Information

   public record BankInfo(String name, String branch) {

   }

  // Enum for Menu Options

   public enum MenuOption {

      OPEN_ACCOUNT,

      DEPOSIT,
```

```
        WITHDRAW,
        TRANSFER,
        EXIT
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankInfo bank = new BankInfo("MiniBank", "Valsad Branch");
      System.out.println("==================================");
        System.out.println("      " + bank.name());
        System.out.println("      " + bank.branch());
        System.out.println("==================================");
        int choice;
do {
            System.out.println("\n===== MINI BANK MENU =====");
            System.out.println("1. Open Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Transfer");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");choice = sc.nextInt();
            String message = switch (choice) {
              case 1 -> MenuOption.OPEN_ACCOUNT + " - To be implemented in a later lab.";
                case 2 -> MenuOption.DEPOSIT + " - To be implemented in a later lab.";
                case 3 -> MenuOption.WITHDRAW + " - To be implemented in a later lab.";
                case 4 -> MenuOption.TRANSFER + " - To be implemented in a later lab.";
                case 5 -> MenuOption.EXIT + " - Thank you for using MiniBank!";
                default -> "Invalid choice! Please try again.";
```

```
        };
 System.out.println(message);
 } while (choice != 5);
       System.out.println("\nGoodbye! Visit MiniBank Again.");
      sc.close();
   }
}
```

```
================================
      MiniBank
      Valsad Branch
================================

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 1
OPEN_ACCOUNT - To be implemented in a later lab.

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 2
DEPOSIT - To be implemented in a later lab.

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 3
WITHDRAW - To be implemented in a later lab.

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 4
TRANSFER - To be implemented in a later lab.

===== MINI BANK MENU =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
EXIT - Thank you for using MiniBank!

Goodbye! Visit MiniBank Again.
```

PRACTICAL 2

In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can

change only through deposit() and withdraw(), never directly. These two classes are the foundationevery later lab extends.

1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods.

2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001").

3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0).

4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods.Do NOT add a public setter for balance.

5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.

CODE:

```
public class MiniBank{
    public static void main(String[] args){
        Customer c1=new Customer("Vidhi","vidhi@gmail.com","9876543210");
        Customer c2=new Customer("Riya","riya@gmail.com","9876501234");
        Customer c3=new Customer("Rahul","rahul@gmail.com","9876512345");
        Account[] accounts=new Account[3];
        accounts[0]=new Account(c1.getName(),5000);
        accounts[1]=new Account(c2.getName(),3000);
        accounts[2]=new Account(c3.getName());
        accounts[0].deposit(1000);
        accounts[0].withdraw(2000);
        accounts[1].deposit(500);
        accounts[1].withdraw(4000);
```

```
        accounts[2].deposit(2000);
        accounts[2].withdraw(500);

        for(int i=0;i<accounts.length;i++){
            System.out.println("Account Number: "+accounts[i].getAccountNumber());
            System.out.println("Owner: "+accounts[i].getOwnerName());
            System.out.println("Balance: Rs."+accounts[i].getBalance());
            System.out.println("Active: "+accounts[i].isActive());
            System.out.println();
        }
    }
}
class Customer{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private static long customerCounter=101;
    private static String generateCustomerId(){
        return "CUST"+customerCounter++;
    }
    public Customer(String name,String email,String mobile){
        this.name=name;
        this.email=email;
        this.mobile=mobile;
        this.customerId=generateCustomerId();
    }
```

```
    public String getName(){
        return name;
    }
    public String getEmail(){
        return email;
    }
    public String getMobile(){
        return mobile;
    }
    public String getCustomerId(){
        return customerId;
    }
}
class Account{
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static int counter=1;
    private static String generateAccountNumber(){
        return String.format("AC%04d",counter++);
    }
    public Account(String ownerName,long openingBalance){
        this.ownerName=ownerName;
        this.balance=openingBalance;
        this.active=true;
```

```
    this.accountNumber=generateAccountNumber();
}
public Account(String ownerName){
    this(ownerName,0);
}
public void deposit(long amount){
    balance+=amount;
}
public boolean withdraw(long amount){
    if(balance>=amount){
        balance-=amount;
        return true;
    }
    return false;
}
public String getAccountNumber(){
    return accountNumber;
}
public String getOwnerName(){
    return ownerName;
}
public long getBalance(){
    return balance;
}
public boolean isActive(){
    return active;
}
```

}

```
Account Number: AC0001
Owner: Vidhi
Balance: Rs.4000
Active: true

Account Number: AC0002
Owner: Riya
Balance: Rs.3500
Active: true

Account Number: AC0003
Owner: Rahul
Balance: Rs.1500
Active: true
```

PRACTICAL 3

In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it.

1. In Account, override the toString() method to return a readable line containing

accountNumber, ownerName and balance.

2. In Account, override equals(Object o) and hashCode() so that two accounts are equal

when their accountNumber values are equal.

3. In Customer, add a public static nested class named Address with String fields line, city

and pincode plus getters; add an Address field to Customer and a getAddress() method.

4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the

customer.

5. In main, print accounts using toString(), compare two Account objects with equals(), and

use instanceof to check an object's type.

CODE:

```
import java.util.Objects;

import java.util.Scanner;
```

```
public class MiniBank{
    public record BankInfo(String name,String branch){}
    public enum MenuOption{
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        DISPLAY,
        EXIT
    }
    static Scanner sc=new Scanner(System.in);
    static Account[] accounts=new Account[10];
    static Customer[] customers=new Customer[10];
    static int count=0;
    public static void main(String[] args){
        BankInfo bank=new BankInfo("MiniBank","Valsad Branch");
        System.out.println("==================================");
        System.out.println(bank.name());
        System.out.println(bank.branch());
        System.out.println("==================================");
        int choice;

        do{
            System.out.println("\n========= MINI BANK =========");
            System.out.println("1. Open Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
```

```
        System.out.println("4. Transfer");
        System.out.println("5. Display Accounts");
        System.out.println("6. Exit");
        System.out.print("Enter Choice : ");
        choice=sc.nextInt();
        sc.nextLine();
        switch(choice){
            case 1->openAccount();
           case 2->deposit();
            case 3->withdraw();
            case 4->transfer();
            case 5->displayAccounts();
           case 6->System.out.println("Thank You For Using MiniBank.");
            default->System.out.println("Invalid Choice");
        }
    }while(choice!=6);
}
public static void openAccount(){
    if(count==accounts.length){
        System.out.println("Bank Account Limit Reached");
        return;
    }
    System.out.print("Enter Name : ");
    String name=sc.nextLine();
    System.out.print("Enter Email : ");
    String email=sc.nextLine();
```

```
        System.out.print("Enter Mobile : ");
        String mobile=sc.nextLine();
        System.out.print("Enter Address Line : ");
        String line=sc.nextLine();
        System.out.print("Enter City : ");
        String city=sc.nextLine();
        System.out.print("Enter Pincode : ");
        String pin=sc.nextLine();
        System.out.print("Enter Opening Balance : ");
        long balance=sc.nextLong();
        sc.nextLine();
        Customer.Address address=new Customer.Address(line,city,pin);
        customers[count]=new Customer(name,email,mobile,address);
        accounts[count]=new Account(name,balance);
        System.out.println("Account Created Successfully");
        System.out.println(accounts[count]);
        count++;
    }
    public static void deposit(){
        System.out.print("Enter Account Number : ");
        String acc=sc.nextLine();
        for(int i=0;i<count;i++){
            if(accounts[i].getAccountNumber().equals(acc)){
                System.out.print("Enter Amount : ");
                long amount=sc.nextLong();
                sc.nextLine();
                accounts[i].deposit(amount);
```

```
            System.out.println("Deposit Successful");
            System.out.println("Balance : Rs."+accounts[i].getBalance());
            return;

        } }
    System.out.println("Account Not Found");
}
public static void withdraw(){
    System.out.print("Enter Account Number : ");
    String acc=sc.nextLine();
    for(int i=0;i<count;i++){
        if(accounts[i].getAccountNumber().equals(acc)){
            System.out.print("Enter Amount : ");
            long amount=sc.nextLong();
            sc.nextLine();
            if(accounts[i].withdraw(amount)){
                System.out.println("Withdrawal Successful");
                System.out.println("Balance : Rs."+accounts[i].getBalance());
            }
            else{
                System.out.println("Insufficient Balance");
            }
            return;
        }
    }
    System.out.println("Account Not Found");
}
```

```
public static void transfer(){
    System.out.print("Enter Sender Account Number : ");
    String from=sc.nextLine();
    System.out.print("Enter Receiver Account Number : ");
   String to=sc.nextLine();
    System.out.print("Enter Amount : ");
    long amount=sc.nextLong();
    sc.nextLine();
    Account sender=null;
    Account receiver=null;
    for(int i=0;i<count;i++){
        if(accounts[i].getAccountNumber().equals(from))
            sender=accounts[i];
        if(accounts[i].getAccountNumber().equals(to))
            receiver=accounts[i];
    }
    if(sender==null||receiver==null){
        System.out.println("Invalid Account Number");
        return;
   }
    if(sender.withdraw(amount)){
        receiver.deposit(amount);
        System.out.println("Transfer Successful");
    }
    else{
        System.out.println("Insufficient Balance");
    }
```

```
    }
    public static void displayAccounts(){
        if(count==0){
            System.out.println("No Accounts Available");
            return;
        }
        for(int i=0;i<count;i++){
            System.out.println(accounts[i]);
            System.out.println("Customer ID : "+customers[i].getCustomerId());
            System.out.println("Email : "+customers[i].getEmail());
            System.out.println("Mobile : "+customers[i].getMobile());
           System.out.println("City : "+customers[i].getAddress().getCity());
            Customer copy=customers[i].clone();
            System.out.println("Clone Created : "+(copy!=null));
            Object obj=accounts[i];
           if(obj instanceof Account){
               System.out.println("instanceof Check Passed");
            }
            System.out.println("-----------------------------------");
        }
    }
    }
class Customer implements Cloneable{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
```

```
    private Address address;
    private static long customerCounter=101;
    private static String generateCustomerId(){
        return "CUST"+customerCounter++;
    }
    public Customer(String name,String email,String mobile,Address address){
        this.name=name;
        this.email=email;
        this.mobile=mobile;
        this.address=address;
        this.customerId=generateCustomerId();
    }
    public String getName(){
        return name;
    }

    public String getEmail(){
        return email;
    }

    public String getMobile(){
        return mobile;
    }
    public String getCustomerId(){
        return customerId;
    }
    public Address getAddress(){
```

```
    return address;
}
@Override
public Customer clone(){
    try{
        return (Customer)super.clone();
    }catch(CloneNotSupportedException e){
        return null;
    }
}
public static class Address{
   private String line;
    private String city;
    private String pincode;
    public Address(String line,String city,String pincode){
        this.line=line;
        this.city=city;
        this.pincode=pincode;
    }
    public String getLine(){
        return line;
    }
    public String getCity(){
        return city;
    }
    public String getPincode(){
        return pincode;
```

```
        }
    }
}
class Account{
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static int accountCounter=1;
    private static String generateAccountNumber(){
        return String.format("AC%04d",accountCounter++);
    }
    public Account(String ownerName,long openingBalance){
        this.ownerName=ownerName;
        this.balance=openingBalance;
        this.active=true;
        this.accountNumber=generateAccountNumber();
    }
    public Account(String ownerName){
        this(ownerName,0);
    }
    public void deposit(long amount){
        if(amount>0){
            balance+=amount;
        }
    }
    public boolean withdraw(long amount){
```

```
        if(amount>0&&balance>=amount){
            balance-=amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber(){
        return accountNumber;
    }
    public String getOwnerName(){
        return ownerName;
    }
    public long getBalance(){
        return balance;
    }
    public boolean isActive(){
        return active;
    }
    @Override
    public String toString(){
        return "Account Number : "+accountNumber+
                "\nOwner Name : "+ownerName+
                "\nBalance : Rs."+balance+
                "\nStatus : "+(active?"Active":"Inactive");
    }
```

```
    @Override
    public boolean equals(Object obj){
        if(this==obj){
            return true;
        }
        if(obj==null||getClass()!=obj.getClass()){
            return false;
        }
        Account a=(Account)obj;
        return accountNumber.equals(a.accountNumber);
    }
    @Override
    public int hashCode(){
        return Objects.hash(accountNumber);
    }
}
```

```
nter Name : riya
nter Email : riyapekh@gmailcom
nter Mobile : 9876543210
nter Address Line : valsad
nter City : gujarat
nter Pincode : 396001
nter Opening Balance : 54601
ccount Created Successfully
ccount Number : AC0002
wner Name : riya
alance : Rs.54601
tatus : Active

======== MINI BANK =========
. Open Account
. Deposit
. Withdraw
. Transfer
. Display Accounts
. Exit
nter Choice : 2
nter Account Number : AC0002
nter Amount : 50000
eposit Successful
alance : Rs.104601

======== MINI BANK =========
. Open Account
. Deposit
. Withdraw
. Transfer
. Display Accounts
. Exit
nter Choice : 3
nter Account Number : AC0002
nter Amount : 50000
ithdrawal Successful
alance : Rs.54601

======== MINI BANK =========
. Open Account
. Deposit
. Withdraw
. Transfer
. Display Accounts
. Exit
nter Choice : 4
nter Sender Account Number : AC0002
nter Receiver Account Number : AC0001
nter Amount : 500000
nsufficient Balance
```

PRACTICAL 4

Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer.

1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String)

and isValidIfsc(String); each returns a boolean.

2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.

3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object.

4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string.

5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.

CODE:

https://github.com/VidhiParekh21/OOPS-A2/tree/main/MiniBank

MiniBank.java

```
import java.util.Scanner;

public class MiniBank {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter correct mobile number: ");

        String mobile1 = sc.nextLine();

        System.out.print("Enter wrong mobile number: ");

        String mobile2 = sc.nextLine();

        System.out.println("Correct mobile: "

                + Validator.isValidMobile(mobile1));
```

```
System.out.println("Wrong mobile: "
        + Validator.isValidMobile(mobile2));
System.out.println();
System.out.print("Enter correct email: ");
String email1 = sc.nextLine();
System.out.print("Enter wrong email: ");
String email2 = sc.nextLine();
System.out.println("Correct email: "
        + Validator.isValidEmail(email1));
System.out.println("Wrong email: "
        + Validator.isValidEmail(email2));
System.out.println();
System.out.print("Enter correct PAN: ");
String pan1 = sc.nextLine();
System.out.print("Enter wrong PAN: ");
String pan2 = sc.nextLine();
System.out.println("Correct PAN: "
        + Validator.isValidPan(pan1));
System.out.println("Wrong PAN: "
        + Validator.isValidPan(pan2));
System.out.println();
System.out.print("Enter correct IFSC: ");
String ifsc1 = sc.nextLine();
System.out.print("Enter wrong IFSC: ");
String ifsc2 = sc.nextLine();
System.out.println("Correct IFSC: "
        + Validator.isValidIfsc(ifsc1));
```

```
        System.out.println("Wrong IFSC: "
                + Validator.isValidIfsc(ifsc2));
        System.out.println();
        System.out.print("Enter command: ");
        String line = sc.nextLine();
        try {
            Command command = CommandParser.parse(line);
            System.out.println("Transaction Type: "
                    + command.type());
            System.out.println("Account Number: "
                    + command.accountNumber());

            System.out.println("Amount: "
                    + command.amount());
        } catch (Exception e) {
            System.out.println("Invalid command.");
        }
        sc.close();
    }
}
```

Customer.java

```
public class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private static long customerCounter = 100;
```

```
    private Address address;
    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }
    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }
    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
    public String getMobile() {
        return mobile;
    }
    public String getCustomerId() {
        return customerId;
    }
    public Address getAddress() {
        return address;
    }
    public void setAddress(Address address) {
```

```
        this.address = address;
    }
    public static class Address {
        private String line;
        private String city;
        private String pincode;
      public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }
        public String getLine() {
            return line;
        }
        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
    }
    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
```

```
            return null;
        }
    }
}
```

Validator.java

```
import java.util.regex.Pattern;

public class Validator {
    private static final Pattern mobile =
            Pattern.compile("^[6-9][0-9]{9}$");
    private static final Pattern email =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
    private static final Pattern pan =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
    private static final Pattern ifsc =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
    public static boolean isValidMobile(String value) {
        return mobile.matcher(value).matches();
    }
   public static boolean isValidEmail(String value) {
        return email.matcher(value).matches();
    }
    public static boolean isValidPan(String value) {
        return pan.matcher(value).matches();
    }
    public static boolean isValidIfsc(String value) {
        return ifsc.matcher(value).matches();
```

```
    }
}
```

TransactionType.java

```
public enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

Command.java

```
public record Command(
        TransactionType type,
        String accountNumber,
        long amount) {
}
```

StatementFormater.java

```
public class StatementFormatter {
    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();
        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ");
        statement.append(account.getAccountNumber());
        statement.append("\n");
        statement.append("Owner Name: ");
        statement.append(account.getOwnerName());
        statement.append("\n");
```

```
        statement.append("Balance: ");
        statement.append(account.getBalance());
        statement.append("\n");
        statement.append("-----------------------------");
                return statement.toString();
    }
}
```

Command Parser.java

```
public class CommandParser {
    public static Command parse(String line) {
        String[] parts = line.split(" ");
        if (parts.length != 3) {
            throw new IllegalArgumentException("Invalid command");
        }
        TransactionType type =
                TransactionType.valueOf(parts[0].toUpperCase());
        String accountNumber = parts[1];
        long amount = Long.parseLong(parts[2]);
        return new Command(type, accountNumber, amount);
    }
}
```

```
Enter correct mobile number: 9999999999
Enter wrong mobile number: 911111111
Correct mobile: true
Wrong mobile: false

Enter correct email: vparekh@gmail.com
Enter wrong email: v@gmai.com
Correct email: true
Wrong email: true

Enter correct PAN: 45698712
Enter wrong PAN: 32165498
Correct PAN: false
Wrong PAN: false

Enter correct IFSC: SBIN0001234
Enter wrong IFSC: SBI123
Correct IFSC: true
Wrong IFSC: false

Enter command: DEPOSIT AC0001 500
Transaction Type: DEPOSIT
Account Number: AC0001
Amount: 500
```

PRACTICAL 5

Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer.

1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean.

2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.

3. Create a class named CommandParser with a public static method parse(String line) that

splits a line such as “DEPOSIT AC0001 500” with split(), converts the parts, and returns

a Command object.

4. Create a class named StatementFormatter with a public static method

buildStatement(Account account) that uses a StringBuilder to assemble a multi-line

account statement string.

5. In main, test each validator with one correct and one wrong input, then parse a sample

command and print its three parts.

CODE:

https://github.com/VidhiParekh21/OOPS-A2/tree/main/MiniBank

MiniBank.java

```
import java.util.Scanner;

public class MiniBank {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter correct mobile number: ");

        String mobile1 = sc.nextLine();

        System.out.print("Enter wrong mobile number: ");

        String mobile2 = sc.nextLine();

        System.out.println("Correct mobile: "

                + Validator.isValidMobile(mobile1));

        System.out.println("Wrong mobile: "

                + Validator.isValidMobile(mobile2));

       System.out.println();

        System.out.print("Enter correct email: ");

        String email1 = sc.nextLine();

        System.out.print("Enter wrong email: ");
```

```
String email2 = sc.nextLine();
System.out.println("Correct email: "
        + Validator.isValidEmail(email1));
System.out.println("Wrong email: "
        + Validator.isValidEmail(email2));
System.out.println();
System.out.print("Enter correct PAN: ");
String pan1 = sc.nextLine();
System.out.print("Enter wrong PAN: ");
String pan2 = sc.nextLine();
System.out.println("Correct PAN: "
        + Validator.isValidPan(pan1));
System.out.println("Wrong PAN: "
        + Validator.isValidPan(pan2));
System.out.println();
System.out.print("Enter correct IFSC: ");
String ifsc1 = sc.nextLine();
System.out.print("Enter wrong IFSC: ");
String ifsc2 = sc.nextLine();
System.out.println("Correct IFSC: "
        + Validator.isValidIfsc(ifsc1));
System.out.println("Wrong IFSC: "
        + Validator.isValidIfsc(ifsc2));
System.out.println();
System.out.print("Enter command: ");
String line = sc.nextLine();
try {
```

```
            Command command = CommandParser.parse(line);
            System.out.println("Transaction Type: "
                    + command.type());
            System.out.println("Account Number: "
                    + command.accountNumber());


            System.out.println("Amount: "
                    + command.amount());
        } catch (Exception e) {
            System.out.println("Invalid command.");
        }
        sc.close();
    }
}
```

Customer.java

```
public class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private static long customerCounter = 100;
    private Address address;
    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }
    public Customer(String name, String email, String mobile) {
```

```
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }
    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
    public String getMobile() {
        return mobile;
    }
    public String getCustomerId() {
        return customerId;
    }
    public Address getAddress() {
        return address;
    }
    public void setAddress(Address address) {
        this.address = address;
    }
    public static class Address {
        private String line;
        private String city;
        private String pincode;
```

```
    public Address(String line, String city, String pincode) {
        this.line = line;
        this.city = city;
        this.pincode = pincode;
    }
    public String getLine() {
        return line;
    }
    public String getCity() {
        return city;
    }

    public String getPincode() {
        return pincode;
    }
  }
  @Override
  public Customer clone() {
    try {
        return (Customer) super.clone();
    } catch (CloneNotSupportedException e) {
        return null;
    }
  }
}
```

Validator.java

```
import java.util.regex.Pattern;

public class Validator {

    private static final Pattern mobile =

            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern email =

            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    private static final Pattern pan =

            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern ifsc =

            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String value) {

        return mobile.matcher(value).matches();

    }

   public static boolean isValidEmail(String value) {

        return email.matcher(value).matches();

    }

    public static boolean isValidPan(String value) {

        return pan.matcher(value).matches();

    }

    public static boolean isValidIfsc(String value) {

        return ifsc.matcher(value).matches();

    }

}
```

TransactionType.java

```
public enum TransactionType {
```

```
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

Command.java

```
public record Command(
        TransactionType type,
        String accountNumber,
        long amount) {
}
```

StatementFormater.java

```
public class StatementFormatter {
    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();
        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ");
        statement.append(account.getAccountNumber());
        statement.append("\n");
        statement.append("Owner Name: ");
        statement.append(account.getOwnerName());
        statement.append("\n");
        statement.append("Balance: ");
        statement.append(account.getBalance());
        statement.append("\n");
        statement.append("----------------------------");
                return statement.toString();
```

```
    }
}
```

Command Parser.java

```
public class CommandParser {
    public static Command parse(String line) {
        String[] parts = line.split(" ");
        if (parts.length != 3) {
            throw new IllegalArgumentException("Invalid command");
        }
        TransactionType type =
                TransactionType.valueOf(parts[0].toUpperCase());
        String accountNumber = parts[1];
        long amount = Long.parseLong(parts[2]);
        return new Command(type, accountNumber, amount);
    }
}
```

```
Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1:
-500
Deposit failed: Deposit amount must be greater than zero.

Enter amount to withdraw from Account 1: 1000
Withdrawal successful.
New Balance: 4000

Enter amount to transfer from Account 1 to Account 2: -200
Transfer failed: Withdrawal amount must be greater than zero.
Transfer operation completed.
Transfer failed: Withdrawal amount must be greater than zero.
Transfer try-catch block finished.
Bank resource is being used.
Bank resource closed automatically.
```

```
Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1:
-500
Deposit failed: Deposit amount must be greater than zero.

Enter amount to withdraw from Account 1: 1000
Withdrawal successful.
New Balance: 4000

Enter amount to transfer from Account 1 to Account 2: -200
Transfer failed: Withdrawal amount must be greater than zero.
Transfer operation completed.
Transfer failed: Withdrawal amount must be greater than zero.
Transfer try-catch block finished.
Bank resource is being used.
Bank resource closed automatically.
```

PRACTICAL 6

With several classes now in place, this lab gives MiniBank a clean structure. You express account capabilities as interfaces, organise the code into model, util and service packages, and build a runnable JAR — the form in which the finished application will actually be delivered and run.

1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it.

2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance.

3. Create a functional interface named WithdrawRule with a single method boolean

allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression.

4. Create a marker interface named Premium (an interface with no methods).

5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import.

6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar.

Code:

Transactable.java

package model;

public interface Transactable { void deposit(long amount);

boolean withdraw(long amount); }

InterestBearing.java

```
package model;


public interface InterestBearing {


    double interestRate();


    long getBalance();


    default double yearlyInterest() {
        return getBalance() * interestRate() / 100;
    }
}
```

Premium.java

```
package model;


public interface Premium {
}
```

WithdrawRule.java

```
package model;


@FunctionalInterface
public interface WithdrawRule {


    boolean allow(Account account, long amount);
}
```

SavingsAccount.java

```
package model;
public class SavingsAccount extends Account implements Premium {
    private long minBalance;
    public SavingsAccount(String ownerName, long balance, long minBalance) {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }
    @Override
    public double interestRate() {
        return 4.0;
    }
    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= minBalance;
    }
}
```

MiniBank.java

```
package service;
import model.Account;
import model.SavingsAccount;
import model.WithdrawRule;
import static java.lang.System.out;
public class MiniBank {

    public static void main(String[] args) {
```

```
Account account =
        new SavingsAccount("Amit", 5000, 1000);


out.println("===== MiniBank =====");


out.println("Account Number: "
        + account.getAccountNumber());
out.println("Owner: "
        + account.getOwnerName());
out.println("Balance: "
        + account.getBalance());
out.println("Interest Rate: "
        + account.interestRate() + "%");
out.println("Yearly Interest: "
        + account.yearlyInterest());
// Anonymous Class
WithdrawRule rule1 = new WithdrawRule() {
    @Override
    public boolean allow(Account account, long amount) {
        return amount <= account.getBalance();
    }
};
// Lambda Expression
WithdrawRule rule2 =
        (acc, amount) -> amount <= acc.getBalance();
long amount = 2000;
out.println("Anonymous Class Result: "
```

```
            + rule1.allow(account, amount));
        out.println("Lambda Result: "
            + rule2.allow(account, amount));
    }
}
```

```
===== MINI BANK =====

Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1: 1000
Deposit successful.
New Balance: 6000

Enter amount to withdraw from Account 1: 500
Withdrawal successful.
New Balance: 5500

===== WITHDRAW RULE TEST =====
Enter amount to check with anonymous rule: 500
Anonymous rule: Withdrawal allowed.
Enter amount to check with lambda rule: 1000
Lambda rule: Withdrawal allowed.

===== YEARLY INTEREST =====
Account 1 yearly interest: 220.0

Enter amount to transfer from Account 1 to Account 2: 500
Transfer successful.
Transfer operation completed.
Account 1 Balance: 5000
Account 2 Balance: 2500
Transfer try-catch block finished.
Bank resource is being used.
Bank resource closed automatically.

C:\Users\maa digital\OneDrive\Desktop\SEMESTER 3\JAVA\JAVA CODES\Mini Bank>
```

PRACTICAL 7

Now that you are comfortable with classes and interfaces, you add metadata to MiniBank.

You define custom annotations (@Id, @Positive, @MaxLength) and read them at run time

with reflection to validate any object automatically — the same idea real frameworks use.

The @Id marker will later identify each record in the generic repository you build in

1. Add the @Override annotation above every overridden method written so far (toString,

equals, hashCode and the subclass methods). Add @FunctionalInterface above the

WithdrawRule interface.

2. In a package named model.annotation, define three custom annotations: @Id (a marker,

with @Target(FIELD) and @Retention(RUNTIME)); @Positive with a member String

message() default "must be > 0"; and @MaxLength with a member int value().

3. Put @Id on the accountNumber field and @Positive on the balance field of Account.

4. Create a class named AnnotationValidator with a public static method String[]

validate(Object obj) that uses reflection — getClass().getDeclaredFields(),

field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the

@Positive and @MaxLength rules and return the error messages as a String[].

5. In main, create an Account with a negative balance and run

AnnotationValidator.validate() to print the error.

CODE:

1. WithdrawRule.java

```
package model;

@FunctionalInterface

public interface WithdrawRule {

    boolean allow(Account account, long amount);

}
```

2. Positive.java

```
package model.annotation;


import java.lang.annotation.Retention;

import java.lang.annotation.RetentionPolicy;
```

```
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Positive {
    String message() default "must be > 0";
}
```

3. Id.java

```
package model.annotation;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Id {
}
```

4. MaxLength.java

```
package model.annotation;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MaxLength {
    int value();
```

```
}
```

5. AnnotationValidator.java

```
package util;
import java.lang.reflect.Field;
import java.util.ArrayList;
import model.annotation.Positive;
import model.annotation.MaxLength;
public class AnnotationValidator {
    public static String[] validate(Object obj) {
        ArrayList<String> errors = new ArrayList<>();
        Class<?> currentClass = obj.getClass();
        while (currentClass != null) {
            Field[] fields = currentClass.getDeclaredFields();
            for (Field field : fields) {
                try {
                    field.setAccessible(true);
                    Object value = field.get(obj);
                    if (field.isAnnotationPresent(Positive.class)) {
                        Positive p =
                            field.getAnnotation(Positive.class);
                        if (value instanceof Number &&
                           ((Number) value).doubleValue() <= 0) {
                            errors.add(
                                field.getName() + ": "
                                + p.message()
                            );
                        }
```

```
            }

            if (field.isAnnotationPresent(MaxLength.class)) {

                MaxLength m =
                    field.getAnnotation(MaxLength.class);

                if (value != null &&
                    value.toString().length() > m.value()) {

                    errors.add(
                        field.getName()
                        + ": maximum length is "
                        + m.value()
                    );
                }
            }
        } catch (Exception e) {
            errors.add(
                "Error reading field: "
                + field.getName()
            );
        }
    }
    currentClass = currentClass.getSuperclass();
}
return errors.toArray(new String[0]);
```

```
    }
}
```

```
Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1: 100
Deposit successful.
New Balance: 5100

Enter amount to withdraw from Account 1: 1333
Withdrawal successful.
New Balance: 3767

===== WITHDRAW RULE TEST =====
Enter amount for anonymous rule: 2000
Anonymous rule: Withdrawal allowed.
Enter amount for lambda rule: 12
Lambda rule: Withdrawal allowed.

===== YEARLY INTEREST =====
Account 1 Interest Rate: 4.0%
Account 1 Yearly Interest: 150.68
Account 2 Interest Rate: 4.0%
Account 2 Yearly Interest: 80.0

Enter amount to transfer from Account 1 to Account 2: 3000
Transfer successful.
Transfer operation completed.
Account 1 Balance: 767
Account 2 Balance: 5000
Transfer operation completed.

===== BANK RESOURCE =====
Bank resource is being used.
Bank resource closed automatically.

===== ANNOTATION VALIDATION =====
Testing account with balance: -500
Validation Errors:
- balance: must be > 0

================================
       MINI BANK FINISHED
================================
```

PRACTICAL 8

Until now a failed withdrawal just returned false. In this lab you make MiniBank fault-tolerant: you replace those flags with a custom exception hierarchy so every failure is explicit and carries information (such as the shortfall). This is the Milestone 1 checkpoint, where the model, validation, annotations and error handling come together.

1. In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall.

2. Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount.

3. Add a method transfer(Account to, long amount) that uses try, catch and finally and re-throws a BankException when needed.

4. In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create.

CODE:

1. BankException.java

Location:

Mini Bank/exception/BankException.java

package exception;

```
public class BankException extends Exception {

    public BankException(String message) {
        super(message);
    }
}
```

2. InsufficientFundsException.java

Location:

Mini Bank/exception/InsufficientFundsException.java

```
package exception;

public class InsufficientFundsException extends BankException {

    private long shortfall;

    public InsufficientFundsException(String message, long shortfall) {
        super(message);
        this.shortfall = shortfall;
    }

    public long getShortfall() {
        return shortfall;
    }
}
```

3. InvalidAmountException.java

```
package exception;
public class InvalidAmountException extends BankException {
    public InvalidAmountException(String message) {
        super(message);
    }
}
```

4. AccountNotFoundException.java

```
package exception;
public class AccountNotFoundException extends BankException {
    public AccountNotFoundException(String message) {
        super(message);
    }
}
```

5. Account.java

```
package model;
import java.util.Objects;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.Positive;
public abstract class Account implements Transactable, InterestBearing {
    @Id
    private final String accountNumber;
    private String ownerName;
    @Positive
```

```
    private long balance;
    private boolean active;
    private static int accountCounter = 0;
    private static String generateAccountNumber() {
        accountCounter++;
        return String.format("AC%04d", accountCounter);
    }
    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
    }
    public Account(String ownerName) {
        this(ownerName, 0);
    }
   @Override
    public void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }
        balance = balance + amount;
    }
    @Override
    public boolean withdraw(long amount)
            throws InsufficientFundsException,
```

```
        InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException(
                "Withdrawal amount must be greater than zero.");
    }

    if (!canWithdraw(amount)) {
        long shortfall = amount - balance;
        throw new InsufficientFundsException(
                "Withdrawal not allowed for this account.",
                shortfall);
    }
    if (amount > balance) {
        long shortfall = amount - balance;
        throw new InsufficientFundsException(
                "Insufficient funds. Shortfall = " + shortfall,
                shortfall);
    }
    balance = balance - amount;
    return true;
}
public void transfer(Account to, long amount)
        throws BankException {
    try {
        this.withdraw(amount);
        to.deposit(amount);
        System.out.println("Transfer successful.");
```

```
        } catch (BankException e) {
            System.out.println(
                    "Transfer failed: " + e.getMessage());
            throw e;
        } finally {
            System.out.println(
                    "Transfer operation completed.");
        }
    }
    public abstract double interestRate();
    public abstract boolean canWithdraw(long amount);
   public String getAccountNumber() {
        return accountNumber;
    }
    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public long getBalance() {
        return balance;
    }
    public boolean isActive() {
        return active;
    }
    public void setActive(boolean active) {
        this.active = active;
```

```
}
@Override
public String toString() {
    return "Account{" +
            "accountNumber='" + accountNumber + '\'' +
            ", ownerName='" + ownerName + '\'' +
            ", balance=" + balance +
            ", active=" + active +
            '}';
}
@Override
public boolean equals(Object o) {

    if (this == o) {
        return true;
    }
    if (!(o instanceof Account)) {
        return false;
    }
    Account account = (Account) o;

    return Objects.equals(
            accountNumber,
            account.accountNumber);
}
@Override
public int hashCode() {
```

```
        return Objects.hash(accountNumber);
    }
}
```

6. BankResource.java

```
package util;
public class BankResource implements AutoCloseable {
    public BankResource() {
    }
    public void showMessage() {
        System.out.println("Bank resource is being used.");
    }
    @Override
    public void close() {
        System.out.println("Bank resource closed automatically.");
    }
}
```

7. MiniBank.java

```
package service;
import java.util.Scanner;
import model.Account;
import util.BankResource;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
public class MiniBank {
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
Account account1 = new Account("Amit", 5000) {
    @Override
    public double interestRate() {
        return 4.0;
    }
    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance();
    }
};
Account account2 = new Account("Riya", 2000) {
    @Override
    public double interestRate() {
        return 4.0;
    }
   @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance();
    }
};
System.out.println("================================");
System.out.println("          MINI BANK");
System.out.println("================================");
System.out.println("\n===== ACCOUNT DETAILS =====");
System.out.println("\nAccount 1:");
System.out.println(account1);
```

```
System.out.println("\nAccount 2:");
System.out.println(account2);
try {
    System.out.print(
            "\nEnter amount to deposit in Account 1: ");
    long amount = sc.nextLong();
    account1.deposit(amount);
    System.out.println("Deposit successful.");
    System.out.println(
            "New Balance: " + account1.getBalance());
} catch (InvalidAmountException e) {
    System.out.println(
            "Deposit failed: " + e.getMessage());

} finally {
    System.out.println(
            "Deposit operation completed.");
}
// Withdrawal
try {
    System.out.print(
            "\nEnter amount to withdraw from Account 1: ");
    long amount = sc.nextLong();
    account1.withdraw(amount);
    System.out.println("Withdrawal successful.");
    System.out.println(
            "New Balance: " + account1.getBalance());
```

```
} catch (InvalidAmountException e) {
    System.out.println(
            "Withdrawal failed: " + e.getMessage());
} catch (InsufficientFundsException e) {
    System.out.println(
            "Withdrawal failed: " + e.getMessage());
    System.out.println(
            "Shortfall: " + e.getShortfall());
} finally {
    System.out.println(
            "Withdrawal operation completed.");
}
try {
    System.out.print(
            "\nEnter amount to transfer "
            + "from Account 1 to Account 2: ");
    long amount = sc.nextLong();
    account1.transfer(account2, amount);
} catch (BankException e) {
    System.out.println(
            "Transfer failed: " + e.getMessage());
} finally {
    System.out.println(
            "Transfer operation completed.");
}
System.out.println("\n===== BANK RESOURCE =====");
 try (BankResource resource = new BankResource()) {
```

```
            resource.showMessage();
        } catch (Exception e) {
            System.out.println(
                    "Resource error: " + e.getMessage());
        }
       System.out.println("\n===== FINAL BALANCES =====");
        System.out.println(
                "Account 1 Balance: "
                + account1.getBalance());
        System.out.println(
                "Account 2 Balance: "
                + account2.getBalance());
        System.out.println("\n================================");
        System.out.println("       MINI BANK FINISHED");
        System.out.println("================================");

        sc.close();
    }
}
```

```
==============================
         MINI BANK
==============================

===== ACCOUNT DETAILS =====
Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1: 100
Deposit successful.
Deposit operation completed.

Enter amount to withdraw from Account 1: 1500
Withdrawal successful.
Withdrawal operation completed.

Enter amount to transfer from Account 1 to Account 2: 500
Transfer successful.
Transfer operation completed.
Transfer operation completed.

===== BANK RESOURCE =====
Bank resource is being used.
Bank resource closed automatically.

===== FINAL BALANCES =====
Account 1 Balance: 3100
Account 2 Balance: 2500

==============================
       MINI BANK FINISHED
==============================
```

```
==============================
          MINI BANK
==============================

===== ACCOUNT DETAILS =====
Account 1:
Account{accountNumber='AC0001', ownerName='Amit', balance=5000, active=true}

Account 2:
Account{accountNumber='AC0002', ownerName='Riya', balance=2000, active=true}

Enter amount to deposit in Account 1:
1000
Deposit successful.
Deposit operation completed.

Enter amount to withdraw from Account 1: 1500
Withdrawal successful.
Withdrawal operation completed.

Enter amount to transfer from Account 1 to Account 2: 10000
Transfer failed: Withdrawal not allowed for this account.
Transfer operation completed.
Transfer failed: Withdrawal not allowed for this account.
Transfer operation completed.

===== BANK RESOURCE =====
Bank resource is being used.
Bank resource closed automatically.

===== FINAL BALANCES =====
Account 1 Balance: 4500
Account 2 Balance: 2000

==============================
       MINI BANK FINISHED
==============================
```

PRACTICAL 9

A real bank serves many users at once. With the single-user logic working, this lab runs many threads against one account, lets you watch the balance come out wrong (a race condition), and then makes deposit and withdraw thread-safe with synchronization — protecting MiniBank's shared data.

1. Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount).

2. In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition.

3. Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct.

4. Print the thread names to observe the thread lifecycle.

CODE:

AccountWorker

```
class AccountWorker implements Runnable {
    private Account account;
    private int times;
    private long amount;
    AccountWorker(Account account, int times, long amount) {
        this.account = account;
        this.times = times;
        this.amount = amount;
    }
    public void run() {
       System.out.println(Thread.currentThread().getName() + " started");
        for (int i = 0; i < times; i++) {
            account.deposit(amount);
```

```
        }
        System.out.println(Thread.currentThread().getName() + " finished");
    }
}
```

MiniBank.java

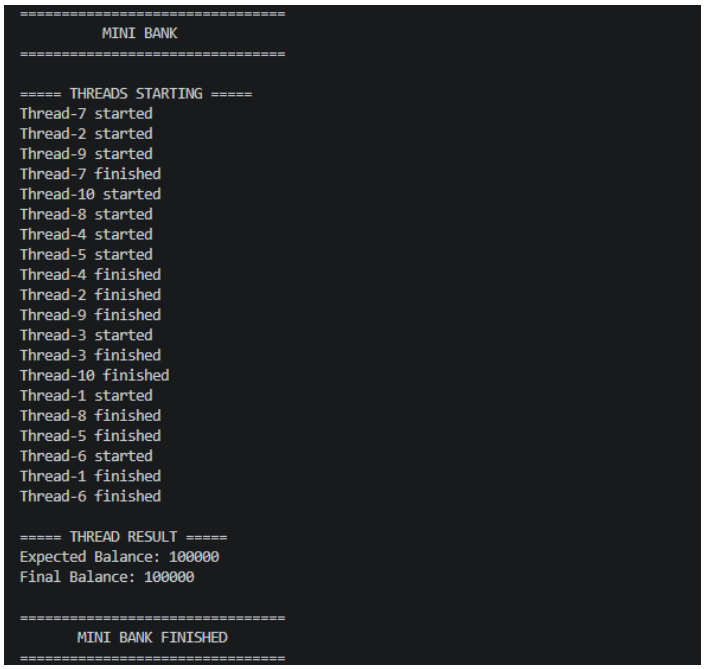
```
package service;
import model.SavingsAccount;
public class MiniBank {
    public static void main(String[] args) {
        System.out.println("================================");
        System.out.println("          MINI BANK");
        System.out.println("================================");
        SavingsAccount account =
                new SavingsAccount("Amit", 0);
        int numberOfThreads = 10;
        int times = 1000;
        long amount = 10;
       Thread[] threads = new Thread[numberOfThreads];
        System.out.println("\n===== THREADS STARTING =====");
        for (int i = 0; i < numberOfThreads; i++) {
            AccountWorker worker =
                    new AccountWorker(account, times, amount);
            threads[i] =
                    new Thread(worker, "Thread-" + (i + 1));
            threads[i].start();
        }
        for (int i = 0; i < numberOfThreads; i++) {
```

```
            try {
                threads[i].join();
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted.");
            }
        }
        long expectedBalance =
                (long) numberOfThreads * times * amount;
        System.out.println("\n===== THREAD RESULT =====");
        System.out.println("Expected Balance: " + expectedBalance);
        System.out.println("Final Balance: " + account.getBalance());
        System.out.println("\n================================");
        System.out.println("       MINI BANK FINISHED");
        System.out.println("================================");
    }}
```

SavingsAccount.java

```
package model;
public class SavingsAccount extends Account {
    private double interestRate = 4.0;
    private long minBalance = 1000;
    public SavingsAccount(String ownerName, long balance) {
        super(ownerName, balance);
    }
    @Override
    public double interestRate() {
        return interestRate;
```

```
    }
    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= minBalance;
    }}
```



PRACTICAL 10

Building on basic threads, this lab gives MiniBank a proper transaction engine. You process a batch of transactions on a managed thread pool, coordinate two threads with wait/notify, and learn to avoid deadlock with consistent lock ordering — the way back-office systems process many transactions safely.

1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls

execute(), and a method stop() that calls shutdown() and awaitTermination().

2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them,

then stop the processor and wait for completion.

3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer

and another thread processes them, using wait() and notify() inside synchronized blocks.

4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first.

CODE:

. TransactionProcessor.java

```
package service;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
public class TransactionProcessor {
    private ExecutorService executor;
    public TransactionProcessor() {
        executor = Executors.newFixedThreadPool(4);
    }
    public void submit(Runnable task) {
        executor.execute(task);
    }
    public void stop() {
        executor.shutdown();

        try {
            executor.awaitTermination(10, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            executor.shutdownNow();
```

```
            Thread.currentThread().interrupt();
        }
    }
}
```

TransactionBuffer.java

```
package service;
public class TransactionBuffer {
    private int[] buffer;
   private int count = 0;
    public TransactionBuffer(int size) {
        buffer = new int[size];
    }
    public synchronized void add(int transaction) {
        while (count == buffer.length) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        buffer[count] = transaction;
        count++;
        System.out.println(
                "Producer added transaction: " + transaction);

        notify();
```

```
    }
    public synchronized int remove() {
        while (count == 0) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
        int transaction = buffer[0];
        for (int i = 1; i < count; i++) {
            buffer[i - 1] = buffer[i];
        }
        count--;
        System.out.println(
                "Consumer processed transaction: " + transaction);
        notify();
        return transaction;
    }
}
```

DeadlockDemo.java

```
package service;
import model.Account;
public class DeadlockDemo {
    public static void transferSafely(
            Account from,
```

```
            Account to,
            long amount) {


        Account first;
        Account second;
        if (from.getAccountNumber()
                .compareTo(to.getAccountNumber()) < 0) {
            first = from;
            second = to;
        } else {
            first = to;
            second = from;
        }
        synchronized (first) {
            synchronized (second) {
                System.out.println(
                        Thread.currentThread().getName()
                        + " safely transferred "
                        + amount);
            }
        }
    }
}
```

MiniBank.java

```
package service;
import model.SavingsAccount;
public class MiniBank {
```

```
public static void main(String[] args) {
    SavingsAccount account1 =
            new SavingsAccount("Amit", 5000);
    SavingsAccount account2 =
            new SavingsAccount("Riya", 5000);
    // Thread Pool

    TransactionProcessor processor =
            new TransactionProcessor();
    processor.submit(
            new TransactionTask(account1, true, 1000));
    processor.submit(
           new TransactionTask(account1, true, 500));
    processor.submit(
           new TransactionTask(account1, false, 200));
    processor.submit(
            new TransactionTask(account2, true, 1000));
    processor.submit(
            new TransactionTask(account2, false, 500));
    processor.stop();
        TransactionBuffer buffer =
            new TransactionBuffer(3);
    Thread producer = new Thread(() -> {
       for (int i = 1; i <= 5; i++) {
            buffer.add(i);
       }
    });
```

```
Thread consumer = new Thread(() -> {
    for (int i = 1; i <= 5; i++) {
       buffer.remove();
    }
});
producer.start();
consumer.start();
try {
    producer.join();
    consumer.join();
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
Thread thread1 = new Thread(() -> {
    DeadlockDemo.transferSafely(
            account1, account2, 100);
});
Thread thread2 = new Thread(() -> {
    DeadlockDemo.transferSafely(
            account2, account1, 200);
});
thread1.start();
thread2.start();
try {
    thread1.join();
    thread2.join();
} catch (InterruptedException e) {
```

```
            Thread.currentThread().interrupt();
        }
    }
}
```

```
==============================
         MINI BANK
==============================

===== TRANSACTION PROCESSOR =====
pool-1-thread-3 withdrew 200
pool-1-thread-2 deposited 500
pool-1-thread-1 deposited 1000
pool-1-thread-4 deposited 1000
pool-1-thread-3 withdrew 500
All transactions completed.

===== PRODUCER CONSUMER =====
Producer added transaction: 1
Producer added transaction: 2
Producer added transaction: 3
Consumer processed transaction: 1
Consumer processed transaction: 2
Consumer processed transaction: 3
Producer added transaction: 4
Producer added transaction: 5
Consumer processed transaction: 4
Consumer processed transaction: 5

===== DEADLOCK PREVENTION =====
Transfer-1 safely transferred 100
Transfer-2 safely transferred 200

==============================
      MINI BANK FINISHED
==============================
```